SLPA — Self-Learning Personal Assistant for Students
UCSC398J

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Computer Science

by

SHANJIVKKUMAR RAMASAMY

23BCS0129

Under the guidance of

Dr. Kavitha B.R

School of Computer Science Engineering and Information Systems

VIT, Vellore

April, 2026

DECLARATION

I hereby declare that the UCSC398J — Project thesis entitled **"SLPA — Self-Learning Personal Assistant for Students"** submitted by me, for the award of the degree of ***Bachelor of Science in Computer Science, School of Computer Science Engineering and Information Systems*** to VIT is a record of Bonafide work carried out by me under the supervision of **Dr. Kavitha B.R, Assistant Professor Sr. Grade 1, SCORE, VIT, Vellore.**

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore
Date:

Signature of the Candidate

SHANJIVKKUMAR RAMASAMY
23BCS0129

CERTIFICATE

This is to certify that the UCSC398J — Project thesis entitled **"SLPA — Self-Learning Personal Assistant for Students"** submitted by Shanjivkkumar Ramasamy (23BCS0129), SCORE, VIT, for the award of the degree of ***Bachelor of Science in Computer Science, School of Computer Science Engineering and Information Systems***, is a record of Bonafide work carried out by him under my supervision during the period 05.12.2025 to 17.04.2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore
Date:

Signature of the Guide
Dr. Kavitha B.R
Assistant Professor Sr. Grade 1, SCORE, VIT, Vellore

Internal Examiner **External Examiner**

Head of the Department
Department of Information Technology

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my UCSC398J — Project guide Dr. Kavitha B.R, Assistant Professor Sr. Grade 1, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore, for her constant guidance, continual encouragement, and unwavering support in my endeavor. My association with her is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of Artificial Intelligence and Educational Technology.

I would like to express my heartfelt gratitude to the Honourable Chancellor Dr. G Viswanathan; respected Vice Presidents Dr. Sankar Viswanathan, Dr. Sekar Viswanathan; Vice Chancellor Dr. V. S. Kanchana Bhaaskaran; Pro-Vice Chancellor Dr. Partha Sharathi Mallick; and Registrar Dr. Jayabarathi T.

My whole-hearted thanks to Dean Dr. Daphne Lopez, School of Computer Science Engineering and Information Systems; Head, Department of Smart Computing, Dr. Vijayan E; SCORE School Project Coordinator Dr. Jayaram Reddy; all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my project successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the project.

Place: Vellore

Shanjivkkumar Ramasamy

EXECUTIVE SUMMARY

The Self-Learning Personal Assistant (SLPA) is an AI-powered academic support platform developed to assist students in managing academic activities and obtaining intelligent contextual guidance. The system integrates a fine-tuned conversational AI model — based on TinyLama optimized with LoRA adapters and trained on approximately 100,000 academic instruction-response pairs — with comprehensive academic data analytics.

Developed using React and TypeScript for the frontend, Supabase for backend services, and Recharts for interactive data visualization, the SLPA platform enables students to ask academic questions, manage study schedules, monitor attendance, and analyse performance trends. The system addresses key gaps in existing educational platforms by integrating conversational AI with academic analytics in a single unified platform.

Key outcomes include an intelligent AI chatbot capable of answering academic queries, a responsive web dashboard, a real-time attendance management system, and a personalized study planner. The system demonstrates how artificial intelligence can transform educational support by providing students with a smart, scalable, and personalized academic companion.

CONTENTS

Chapter / Section	Page No.
Acknowledgement	**i**
Executive Summary	**ii**
Table of Contents	**iii**
List of Figures	**vi**
List of Tables	**vii**
List of Abbreviations	**viii**
Symbols and Notations	**ix**
Chapter 1 Introduction	**1**
1.1 Objective	1
1.2 Motivation	2
1.3 Background	3
Chapter 2 Dissertation Description and Goals	**4**
2.1 Problem Statement	4
2.2 Scope of the Project	4
2.3 Gap Analysis	5
2.4 Proposed System	5
Chapter 3 Technical Specification	**7**
3.1 Technology Stack	7
3.2 Functional Requirements	8
3.3 Non-Functional Requirements	9
3.4 Hardware Requirements	9
Chapter 4 Design Approach and Details	**10**
4.1 Literature Survey	10

4.2 Findings from Literature Survey 11
4.3 System Architecture 12
4.3.1 Presentation / Frontend Layer 14
4.3.2 Application / API Communication Layer 14
4.3.3 AI Processing Layer 14
4.3.4 Backend Layer 14
4.3.5 Data Layer 15
4.4 UML Diagrams 15
4.5 System Modules 17
4.6 LLM Training Pipeline 17
4.7 Codes and Standards 18

Chapter 5 Schedule, Tasks and Milestones 19
5.1 Project Timeline 19
5.2 Phase-wise Description 19

Chapter 6 Dissertation Demonstration 21
6.1 Implementation Overview 21
6.2 Sample Code 21
6.2.1 Model Loading with LoRA Fine-Tuning 21
6.2.2 AI Response Generation 22
6.3 System Screenshots 22

Chapter 7 Result and Discussion 25
7.1 AI Model Performance Evaluation 25
7.2 System Usability Testing 26
7.3 Cost Analysis 26
7.4 Discussion 27

Chapter 8 Summary 28
8.1 Summary of Findings 28

8.2 Recommendations 28
8.3 Conclusion 29

Chapter 9 References 31

Appendix A --

LIST OF FIGURES

Figure No.	Title	Page No.
Fig. 4.1	SLPA System Architecture Diagram	13
Fig. 4.2	UML Diagrams — SLPA System	16
Fig. 6.1	Login and Registration	22
Fig. 6.2	Registration Page	22
Fig. 6.3	Academic Performance Dashboard — Analytics Visualization	23
Fig. 6.4	Academic Timetable	23
Fig. 6.5	AI Chatbot	24
Fig. 6.6	Study Planner	24

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Gap Analysis — Existing Systems vs. SLPA	5
Table 3.1	Technology Stack — SLPA System	7
Table 3.2	Functional Requirements Specification	8
Table 3.3	Non-Functional Requirements	9
Table 4.1	Literature Survey	10
Table 4.2	SLPA System Modules and Features	17
Table 5.1	Project Schedule and Milestones	19
Table 7.1	AI Model Performance Metrics — Before and After Fine-Tuning	25
Table 7.2	Project Cost Analysis	26

LIST OF ABBREVIATIONS

Abbreviation	Expanded Form
AI	Artificial Intelligence
API	Application Programming Interface
BLEU	Bilingual Evaluation Understudy
CSS	Cascading Style Sheets
DB	Database
ERP	Enterprise Resource Planning
GPU	Graphics Processing Unit
HTTPS	Hypertext Transfer Protocol Secure
ITS	Intelligent Tutoring System
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
LoRA	Low-Rank Adaptation
LMS	Learning Management System
MIT	Massachusetts Institute of Technology
NLP	Natural Language Processing
NLU	Natural Language Understanding
PWA	Progressive Web Application
REST	Representational State Transfer
SLPA	Self-Learning Personal Assistant
SRS	Software Requirements Specification
SUS	System Usability Scale
SQL	Structured Query Language
TypeScript	TypeScript (typed superset of JavaScript)
UI	User Interface
URL	Uniform Resource Locator
UML	Unified Modeling Language
VIT	Vellore Institute of Technology
XSS	Cross-Site Scripting

SYMBOLS AND NOTATIONS

Symbol / Notation	Description
r	LoRA rank — controls the number of trainable parameters in adaptation matrices
α (alpha)	LoRA scaling factor — controls the magnitude of LoRA weight updates
W	Pre-trained weight matrix of the base language model
ΔW	Low-rank update matrix added during LoRA fine-tuning ($\Delta W = BA$)
A, B	Low-rank decomposition matrices where $A \in R^{(d \times r)}$ and $B \in R^{(r \times k)}$
d	Dimension of model hidden state (embedding size)
k	Output dimension of the weight matrix
N	Total number of training samples in the dataset
T	Sequence length (maximum token length = 512)
P(y|x)	Conditional probability of output sequence y given input x (LLM output)
θ	Model parameters (weights) of the fine-tuned Phi-3 Mini language model
L	Training loss function (cross-entropy loss for causal language modeling)
η	Learning rate used during LoRA fine-tuning ($\eta = 2 \times 10^{-4}$)
n	Number of concurrent users supported by the system
t_r	System response time (target: $t_r < 2000$ ms for 95% of requests)
f(x)	AI model inference function mapping query x to response y
%A	Attendance percentage for a subject = (classes attended / total classes) × 100
GPA	Grade Point Average — weighted mean of grade points across all subjects
BLEU	Bilingual Evaluation Understudy score — measures AI response quality (0 to 1)

CHAPTER 1

INTRODUCTION

Artificial Intelligence has emerged as one of the most transformative technologies in the twenty-first century, permeating virtually every domain of human activity including education, healthcare, finance, and governance. The rapid proliferation of Large Language Models (LLMs) and AI-driven platforms has redefined how students interact with knowledge, how educators deliver instruction, and how institutions monitor academic progress. In the context of higher education, students face increasing challenges related to time management, academic performance monitoring, and access to timely academic guidance.

The Self-Learning Personal Assistant (SLPA) is a smart, web-based academic support system designed specifically for college students. It integrates cutting-edge AI technology with practical academic management tools to create an intelligent, unified platform. By combining a fine-tuned Large Language Model with real-time academic data analytics, SLPA addresses critical gaps that exist in traditional learning management systems.

Unlike conventional academic tools that operate in silos — offering either AI assistance or academic tracking, but not both — SLPA provides a seamless integration of conversational AI, attendance monitoring, timetable management, study planning, and performance analytics within a single web application. This holistic approach ensures that students have access to a comprehensive academic companion that adapts to their unique needs and learning patterns.

1.1 Objective

The primary objective of this project is to develop an AI-powered academic assistant, SLPA, that leverages machine learning and natural language processing to support students

in managing academic activities and obtaining intelligent academic guidance. The specific objectives of the project are:

To develop an AI-powered academic assistant capable of answering student queries in real time. To implement a conversational chatbot using a fine-tuned Phi-3 Mini language model trained on academic instruction-response data. To design and develop a responsive web interface using React and TypeScript. To integrate a secure backend using Supabase for user authentication and data management. To provide comprehensive data visualization for academic performance insights using Recharts. To enable attendance tracking, study planning, and timetable management within the platform. To demonstrate the effectiveness of parameter-efficient fine-tuning techniques such as LoRA in educational AI applications.

1.2 Motivation

The motivation for developing SLPA arises from the observation that students in modern higher education institutions face a multitude of challenges that go unaddressed by existing systems. Many students struggle with managing attendance across multiple subjects, planning their study schedules efficiently, and seeking timely clarification for academic doubts. Traditional learning management systems lack intelligent interaction capabilities, and generic AI assistants are not tailored to academic contexts.

Moreover, the advancement of lightweight language models such TinyLama has made it feasible to develop domain-specific AI assistants without the prohibitive computational costs associated with larger models. The ability to fine-tune such models using parameter-efficient techniques like Low-Rank Adaptation (LoRA) has opened new possibilities for creating intelligent, affordable, and accurate academic assistants. SLPA is motivated by the vision of democratizing access to intelligent academic guidance for all students.

1.3 Background

The concept of Intelligent Tutoring Systems (ITS) has been explored since the 1970s, with early systems such as SOPHIE and GUIDON demonstrating the potential of AI-driven educational support. However, these early systems were limited in scope and accessibility. The emergence of the World Wide Web and the subsequent proliferation of online learning platforms transformed the educational landscape, enabling large-scale deployment of digital learning tools.

Recent years have witnessed an exponential growth in the capabilities of Large Language Models, with models such as GPT, LLaMA, and Phi demonstrating remarkable proficiency in natural language understanding and generation. These developments have revitalized interest in AI-based educational assistants. The SLPA project builds upon this foundation by integrating modern LLM technology with practical academic management features to create a holistic student support system.

CHAPTER 2

DISSERTATION DESCRIPTION AND GOALS

The SLPA project represents a comprehensive dissertation in the domain of Educational Artificial Intelligence. This chapter describes the detailed goals, scope, and intended outcomes of the project, providing a clear roadmap of what the system aims to achieve and the boundaries within which it operates.

2.1 Problem Statement

Students often face difficulties in accessing timely academic support, managing study schedules, and analysing their academic progress. Existing learning systems primarily focus on static resources and lack intelligent interaction capabilities. Although intelligent tutoring systems exist, many suffer from limitations such as limited conversational abilities, poor contextual understanding, and a lack of integration with academic data analytics.

Additionally, students require tools that can provide personalized learning assistance, academic insights, and automated query responses within a single platform. The fragmentation of academic tools across multiple applications creates cognitive overhead and reduces student productivity. There is, therefore, a clear need for an intelligent, unified system that combines conversational AI, academic analytics, and personalized recommendations to support student learning more effectively.

2.2 Scope of the Project

The scope of the SLPA system includes the development of a web-based academic assistant that provides intelligent responses and academic insights to students. The system is designed to serve college students across disciplines within the academic year 2025-2026 at VIT, Vellore, with the potential for broader institutional deployment.

The system focuses on academic query answering using a fine-tuned language model, student data management including attendance and study schedules, academic performance analysis through interactive dashboards, personalized study recommendations, and integration of conversational AI with educational datasets. The project is deliberately limited to academic assistance functionalities and is not intended to replace full institutional learning management systems.

2.3 Gap Analysis

Table 2.1: Gap Analysis — Existing Systems vs. SLPA

S.No	Gap Identified	Existing System Limitation	SLPA Solution
1	**Lack of Integrated Platforms**	Tutoring, attendance, and AI systems exist separately	SLPA unifies all academic tools in one platform
2	**Limited Personalization**	Generic AI responses without student-specific context	AI responses integrate attendance and schedule data
3	**Dependency on External APIs**	Latency, cost, and data privacy risks with external AI APIs	Fine-tuned local LLM reduces external dependencies
4	**Absence of Domain-Specific LLM**	General-purpose models provide inaccurate academic responses	TinyLama fine-tuned on 100K academic instruction pairs
5	**Lack of Data-Driven Insights**	No integration of AI interaction with performance analytics	Analytics dashboard combined with AI recommendations

2.4 Proposed System

The proposed SLPA system integrates multiple technologies to deliver a comprehensive, intelligent academic assistance platform. The system is architected around four core components: a React and TypeScript frontend for intuitive user interaction, a Supabase backend for secure authentication and data management, a fine-tuned Phi-3 Mini language model for intelligent academic query processing, and a Recharts-based analytics module for performance visualization.

The architecture ensures modularity, allowing individual components to be upgraded or scaled independently. Students can interact with the AI chatbot, monitor their

attendance, manage study plans, and review performance dashboards, all from within a single, responsive web interface. The system is designed with security, scalability, and usability as core non-functional priorities.

CHAPTER 3

TECHNICAL SPECIFICATION

This chapter presents the detailed technical specifications of the SLPA system, encompassing the hardware and software requirements, the technology stack, and the functional and non-functional requirements that govern system design and implementation.

3.1 Technology Stack

Table 3.1: Technology Stack — SLPA System

Component	Technology	Version / Details	Purpose
Frontend	React	React 18 with TypeScript	User interface and component management
Language	TypeScript	TypeScript 5.x (strict mode)	Type-safe application development
Styling	Tailwind CSS	Tailwind CSS 3.x	Responsive and utility-first UI design
Backend	Supabase	Supabase (PostgreSQL + Auth)	Authentication, database, and API services
AI Model	TinyLama	TinyLlama/TinyLlama-1.1B-Chat-v1.0	Base language model for academic query processing
Fine-Tuning	LoRA + Unsloth	4-bit quantization, 100K training pairs	Parameter-efficient LLM fine-tuning
Training Env.	Google Colab	GPU-accelerated (T4)	Model training and evaluation environment
Visualization	Recharts	Recharts 2.x	Interactive academic performance charts
Version Control	Git / GitHub	GitHub repository	Source code management and collaboration

3.2 Functional Requirements

Table 3.2: Functional Requirements Specification

ID	Requirement	Description	Priority
FR1	User Registration & Login	Secure authentication with email and password via Supabase Auth	High
FR2	Student Profile Management	Manage academic profile including name, department, and student ID	High
FR3	AI Chatbot Interaction	Students interact with fine-tuned TinyLama model for academic queries	High
FR4	Attendance Tracking	Record, update, and view subject-wise attendance with percentage analytics	High
FR5	Study Planner Management	Create, edit, and track study tasks with deadlines and priority levels	Medium
FR6	Timetable Management	View and manage weekly class schedules with conflict alerts	Medium
FR7	Performance Analytics	Interactive Recharts dashboards showing GPA trends and academic progress	Medium
FR8	Personalized Recommendations	AI-generated study tips and recommendations based on academic performance	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

Attribute	Requirement	Measurement Criteria
Performance	System must respond to user queries within 2 seconds	Response time < 2s for 95% of requests
Security	User data and authentication must be secured using industry standards	JWT-based auth; all data encrypted in transit via HTTPS
Scalability	System should support concurrent access by multiple users	Supabase infrastructure supports horizontal scaling
Reliability	System must maintain consistent AI response accuracy	AI model accuracy > 85% on academic query benchmarks
Usability	Interface must be intuitive and accessible for all students	System Usability Scale (SUS) score > 80
Availability	Platform should be available 24/7 with minimal downtime	99.5% uptime via cloud-hosted Supabase backend

3.4 Hardware Requirements

The SLPA system is designed to be accessible on standard consumer hardware. For end users, a modern web browser with internet connectivity on any device — desktop, laptop, tablet, or mobile phone — is sufficient. For development and model training, the following hardware specifications were used: a system with Intel Core i7 or equivalent processor, 16 GB RAM, and a dedicated GPU (NVIDIA T4 via Google Colab) for model fine-tuning. The backend is hosted on Supabase's cloud infrastructure, eliminating the need for dedicated server hardware.

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 Literature Survey

A comprehensive literature review was conducted to understand the state-of-the-art in AI-based educational systems, intelligent tutoring platforms, and academic analytics. The following table presents the ten key research papers reviewed, along with their merits and limitations:

Table 4.1: Literature Survey

S.No	Title	Author & Year	Publisher	Merits	Demerits
1	Improving adaptive support in intelligent tutoring systems through learning analytics	Aleven et al., 2022	IEEE Transactions on Learning Technologies	Uses learning analytics to personalize instruction; improves learning outcomes; supports adaptive feedback mechanisms	Requires continuous data collection; high computational overhead; dependent on data quality
2	Supporting natural language interaction in conversational intelligent tutoring systems	Arevalillo-Herraez et al., 2023	IEEE Transactions on Learning Technologies	Enables interactive learning through NLP; improves student engagement; supports real-time query handling	Limited understanding of complex queries; struggles with ambiguous language; requires frequent model updates
3	Educational data mining and learning analytics: Recent advances and challenges	Romero & Ventura, 2023	Elsevier — Expert Systems with Applications	Identifies learning patterns; supports decision-making; improves personalization strategies	Data privacy concerns; requires large datasets; complex preprocessing
4	Roles and research trends of artificial intelligence in education	Hwang & Tu, 2022	Elsevier — Computers and Education: Artificial Intelligence	Provides comprehensive overview of AI in education; supports learner autonomy; promotes adaptive learning	Lacks implementation details; ethical and bias-related concerns
5	Personalized adaptive learning in higher	Du Plooy et al., 2024	Elsevier — Heliyon	Highlights effectiveness of adaptive learning; supports mobile learning	High implementation cost;

S.No	Title	Author & Year	Publisher	Merits	Demerits
	education: A scoping review			adoption; improves accessibility	infrastructure dependency; scalability issues
6	Artificial intelligence in personalized learning: A global systematic review	Fortuna et al., 2025	Elsevier — Social Sciences & Humanities Open	Demonstrates global effectiveness of AI-driven learning; improves efficiency; supports diverse learners	Requires advanced technical expertise; high computational and maintenance costs
7	Trustworthy knowledge tracing models for personalized tutoring systems	Yu et al., 2024	IEEE Transactions on Learning Technologies	Improves accuracy of performance prediction; enhances trust in AI models; supports adaptive tutoring	Model complexity; increased training time; limited interpretability
8	Adaptive prompt design for intelligent tutoring systems	Liu et al., 2024	IEEE Transactions on Learning Technologies	Enhances response relevance; improves learner satisfaction; supports adaptive questioning	High computational requirements; sensitive to prompt design errors
9	Generative data augmentation for educational datasets	Zhang et al., 2025	IEEE Transactions on Learning Technologies	Addresses data sparsity; improves model robustness; enhances prediction accuracy	Risk of synthetic data bias; validation complexity; increased processing time
10	AI-driven academic assistants for mobile learning	Arnau-González et al., 2025	Elsevier — Expert Systems with Applications	Enables personalized learning on mobile platforms; improves flexibility and accessibility	Internet dependency; data security risks; limited offline functionality

4.2 Findings from Literature Survey

The literature review indicates that AI-based tutoring systems significantly improve personalized learning experiences and student engagement. Research consistently highlights the importance of natural language interaction, learning analytics, and adaptive learning techniques in modern educational platforms. Studies by Aleven et al. (2022) and Arevalillo-Herraez et al. (2023) demonstrate the effectiveness of adaptive support mechanisms and NLP-based conversational systems in improving student outcomes.

However, many existing systems face challenges such as limited contextual understanding, high computational requirements, and a lack of integration with academic data management systems. Recent advancements in Large Language Models, particularly lightweight models such as Phi-3 Mini, have substantially improved conversational capabilities, making them suitable for educational assistance applications without requiring enterprise-grade infrastructure. The SLPA system directly addresses these identified limitations.

4.3 System Architecture

The SLPA system is designed using a layered architecture pattern that ensures separation of concerns, modularity, and scalability. The architecture comprises seven distinct layers, each responsible for specific system functions. The overall system architecture is illustrated in Figure 4.1 below.

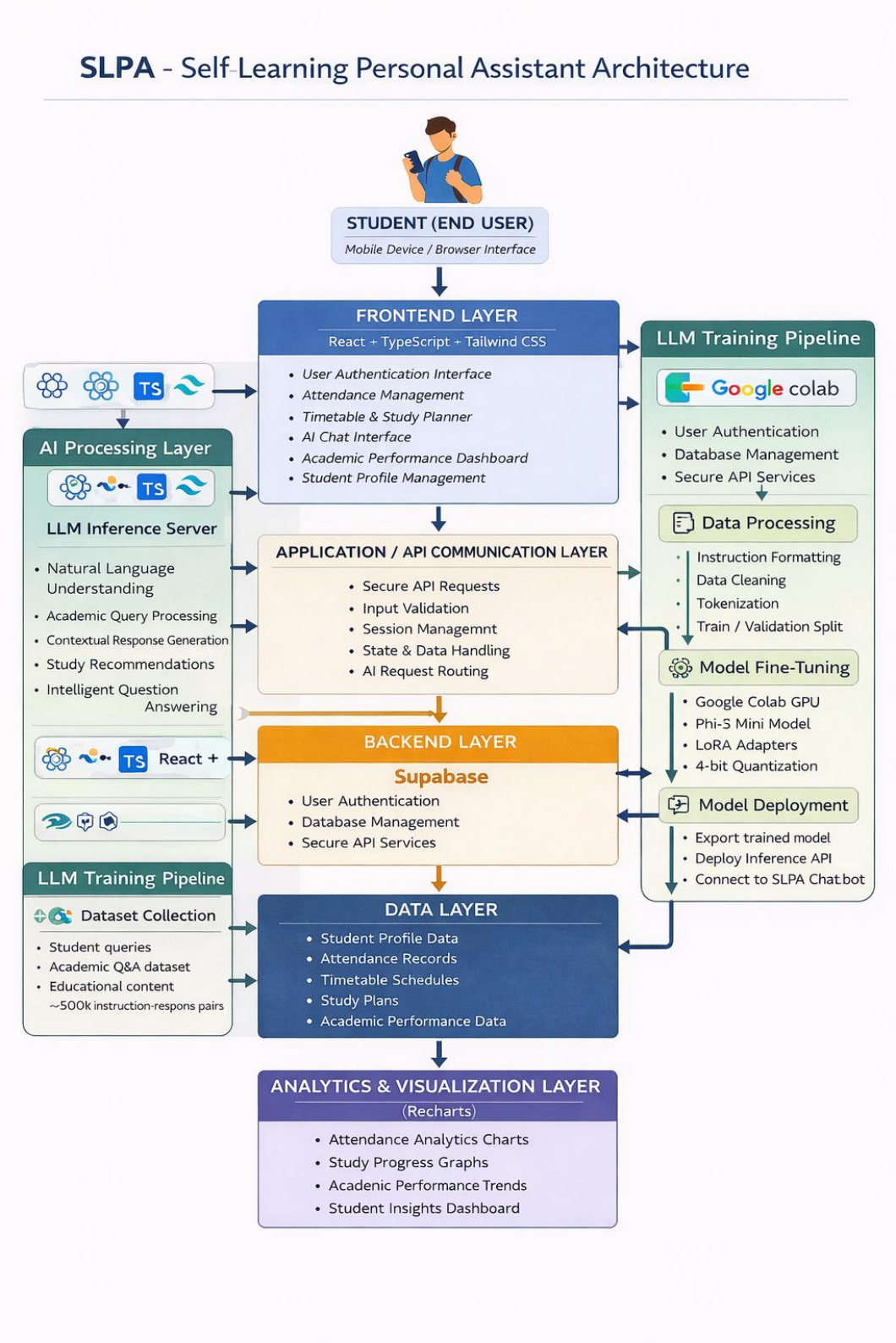

Fig. 4.1 SLPA System Architecture Diagram

4.3.1 Presentation / Frontend Layer

The Presentation Layer acts as the primary user interface through which students interact with the SLPA system. It is implemented using React 18, TypeScript, and Tailwind CSS to provide a responsive, performant, and interactive user experience. This layer includes modules for user authentication, attendance management, timetable and study planner, AI chat interface, academic performance dashboard, and student profile management. All interactions are forwarded to the application communication layer through secure API calls.

4.3.2 Application / API Communication Layer

The Application Communication Layer acts as middleware connecting the frontend with backend services and AI components. Its primary responsibilities include secure API request handling, input validation and request filtering, session management, data processing and routing, and AI request forwarding to the inference engine. This layer ensures that all user requests are validated and securely processed before interacting with the backend database or AI system.

4.3.3 AI Processing Layer

The AI Processing Layer is responsible for handling student queries using the fine-tuned Large Language Model. The LLM Inference Server within this layer performs natural language understanding, academic query processing, context-aware response generation, study recommendations, and intelligent question answering. The system utilizes a fine-tuned version of TinyLama, optimized using LoRA adapters and 4-bit quantization for efficient inference on standard hardware.

4.3.4 Backend Layer

The Backend Layer manages server-side logic, authentication, and database interactions. Implemented using Supabase, the backend provides secure user authentication via JWT tokens, PostgreSQL database management, RESTful API endpoints for data retrieval and

updates, and comprehensive student data management. Supabase's real-time capabilities also enable live updates to attendance and schedule data.

4.3.5 Data Layer

The Data Layer stores all structured academic and student-related information used by the system. The PostgreSQL database managed through Supabase stores student profile data, attendance records, timetable schedules, study plans, and academic performance data. This data is accessed by both backend services and the AI system to provide context-aware recommendations and personalized analytics.

4.4 UML Diagrams

The system design is further elaborated through Unified Modeling Language (UML) diagrams that capture the structural and behavioral aspects of the SLPA platform. The UML diagrams include use case diagrams, class diagrams, sequence diagrams, and activity diagrams as illustrated in Figure 4.2.

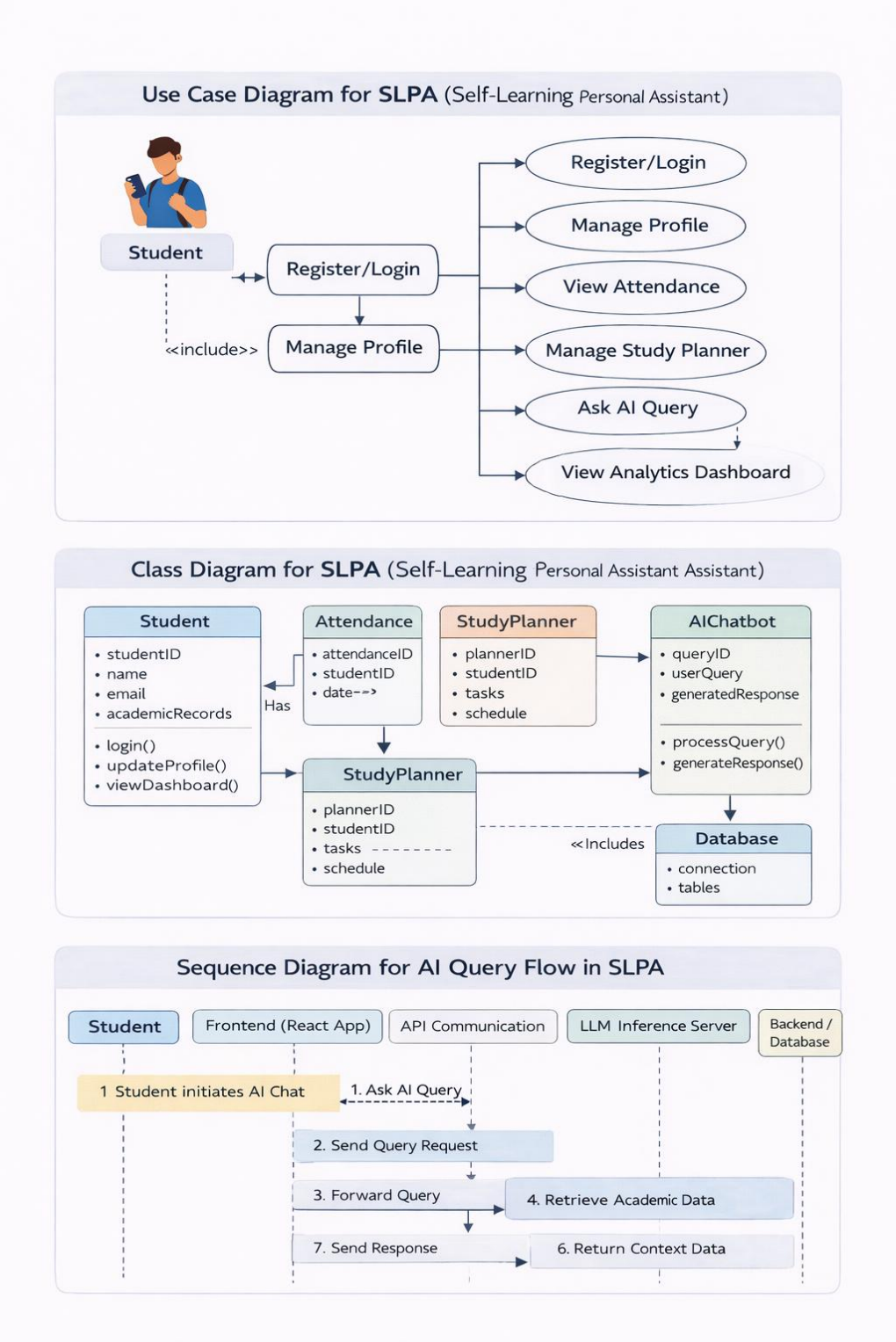

Fig. 4.2 UML Diagrams — SLPA System

4.5 System Modules

Table 4.2: SLPA System Modules and Features

Module	Features	Description
Authentication	Login, Registration, Session	Secure user authentication using Supabase Auth with JWT tokens and session management
AI Chatbot	Query Processing, NLP, Recommendations	Fine-tuned TinyLama model handles academic queries and generates context-aware responses
Attendance Tracker	Daily Records, Analytics, Alerts	Students can log and monitor attendance with visual percentage indicators and subject-wise breakdowns
Study Planner	Task Scheduling, Reminders, Progress	Intelligent study schedule management with priority-based task assignment and completion tracking
Timetable	Weekly Schedule, Conflict Detection	Interactive timetable management with class schedule visualization and conflict detection
Performance Dashboard	Charts, Trends, Insights	Recharts-based interactive dashboards displaying academic performance trends and GPA analytics
Profile Management	User Data, Settings, Preferences	Student profile management including academic details, preferences, and notification settings

4.6 LLM Training Pipeline

The LLM Training Pipeline is a critical architectural component responsible for preparing and fine-tuning the TinyLama language model for academic domain specialization. The pipeline consists of four sequential stages.

Dataset Collection: The training dataset contains approximately 100,000 instruction-response pairs collected from student academic queries, educational Q&A datasets, and academic learning resources. This large and diverse dataset ensures that the model develops comprehensive academic domain knowledge.

Data Processing: Data preparation involves instruction-response formatting using the Alpaca format, data cleaning and normalization to remove inconsistencies, tokenization using the TinyLama tokenizer with a maximum sequence length of 512 tokens, and stratified splitting into training (90%) and validation (10%) sets.

Model Fine-Tuning: The base TinyLama model is fine-tuned using LoRA adapters with rank r=16 and alpha=32 applied to the attention layers. 4-bit quantization using BitsAndBytes reduces memory footprint by approximately 75%, enabling training on free Google Colab GPUs.

Model Deployment: After training, the fine-tuned model is deployed as a FAST API inference service, and integrated with the SLPA chatbot interface through secure HTTP endpoints.

4.7 Codes and Standards

The SLPA project adheres to established software development standards and best practices. The frontend follows the React component architecture pattern with TypeScript strict mode enabled for type safety. Code formatting is enforced using ESLint and Prettier with a consistent style guide. The backend API follows FASTAPI design principles with proper HTTP status codes and JSON response formats. Database design follows third normal form (3NF) to eliminate data redundancy. Security standards include HTTPS for all communications, JWT-based authentication with proper token expiry, and input sanitization to prevent SQL injection and XSS attacks.

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

5.1 Project Timeline

The SLPA project was developed over a structured period spanning December 2025 to April 2026, organized into six distinct phases. The development followed an agile-inspired iterative approach, with continuous review and refinement at each stage.

Table 5.1: Project Schedule and Milestones

Phase	Activity	Duration	Deliverable	Status
1	Research & Requirement Analysis	Dec 2025 – Jan 2026	SRS Document, Gap Analysis	Completed
2	System Design	Jan 2026	Architecture Diagrams, DB Schema	Completed
3	LLM Dataset & Fine-Tuning	Jan – Feb 2026	Fine-Tuned Phi-3 Mini Model	Completed
4	Frontend Development	Feb 2026	React + TypeScript Web App	Completed
5	Backend & AI Integration	Feb – Mar 2026	Supabase Backend, API Integration	Completed
6	Testing & Optimization	Mar 2026	Test Reports, Optimized System	Completed
7	Documentation & Submission	Apr 2026	Final Project Report	Completed

5.2 Phase-wise Description

Phase 1 — Research and Requirement Analysis (December 2025): Conducted a comprehensive study of intelligent tutoring systems, AI academic assistants, and educational analytics platforms. Performed gap analysis on existing systems and defined functional and non-functional requirements for SLPA. Key deliverables include the Software Requirements Specification (SRS) document and the gap analysis report.

Phase 2 — System Design (January 2026): Designed the layered system architecture, database schema with entity-relationship diagrams, user interface wireframes, and UML behavioral diagrams including use case, class, sequence, and activity diagrams. The architecture decisions made in this phase guided all subsequent development activities.

Phase 3 — LLM Dataset and Fine-Tuning (January – February 2026): Curated and preprocessed the 100,000 instruction-response pair dataset. Configured and executed the LoRA fine-tuning pipeline using Unsloth and Google Colab. Evaluated model performance using BLEU score, response accuracy, and response time benchmarks before declaring the model ready for integration.

Phase 4 — Application Development (February 2026): Developed the React and TypeScript frontend with all seven major modules. Configured Supabase backend with PostgreSQL database schema, authentication flows, and FASTAPI endpoints. Implemented the Recharts-based analytics dashboard with interactive attendance and performance charts.

Phase 5 — Testing and Optimization (March 2026): Conducted comprehensive functional testing of all system modules. Evaluated AI response quality, latency, and accuracy. Optimized system performance through API call batching, React component memoization, and database query optimization. User acceptance testing was performed with a cohort of students.

Phase 6 — Documentation and Submission (April 2026): Prepared comprehensive project documentation including system design documents, API documentation, user manuals, and this final project report. The system was finalized and submitted for academic evaluation.

CHAPTER 6

DISSERTATION DEMONSTRATION

This chapter presents the implementation details and visual demonstration of the SLPA system. The screenshots below illustrate the key functional modules of the platform, showcasing the user interface design, AI chatbot interaction, attendance management, and academic performance dashboards.

6.1 Implementation Overview

The SLPA system implementation integrates frontend development, backend services, and AI model deployment into a cohesive and functional platform. The user interface is developed using React 18 with TypeScript, ensuring type safety and component reusability. Backend services are implemented using Supabase, which provides PostgreSQL database management, JWT-based authentication, and real-time API capabilities.

The AI component is built upon a fine-tuned Phi-3 Mini model trained on academic datasets using LoRA adapters and deployed as a REST API inference service. Interactive dashboards are implemented using Recharts to visualize attendance and academic performance trends in real time. The system follows a component-based architecture on the frontend, with clearly defined service modules for API communication.

6.2 Sample Code

The following code snippets illustrate key implementation aspects of the SLPA system.

6.2.1 Model Loading with LoRA Fine-Tuning (Unsloth)

```
from unsloth import FastLanguageModel
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/ TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    max_seq_length=512,
    load_in_4bit=True)
```

6.2.2 AI Response Generation (Inference)

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
def generate_response(query):
    inputs = tokenizer(query, return_tensors="pt").to("cuda")
    outputs = model.generate(**inputs, max_new_tokens=200)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

6.3 System Screenshots

The following figures present screenshots of the SLPA system across its key functional modules, demonstrating the user interface design and system functionality.

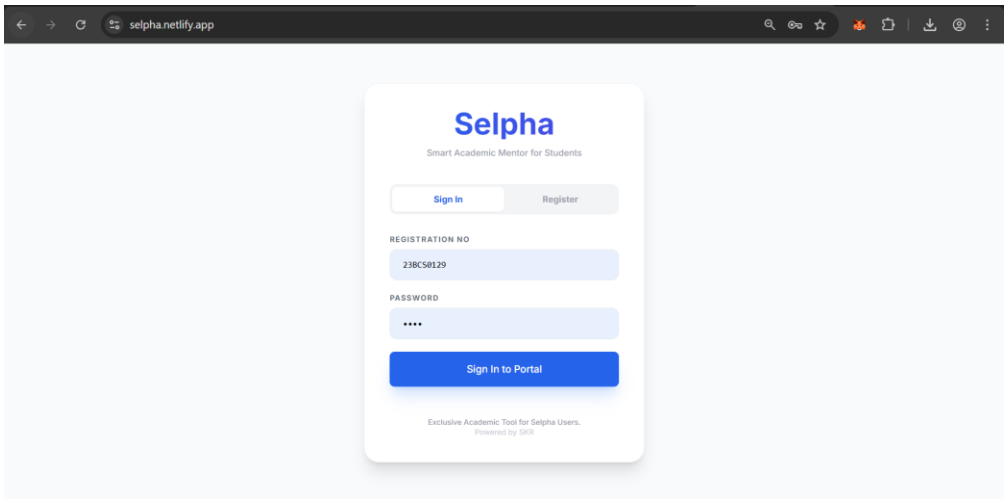

Fig. 6.1 Login Page

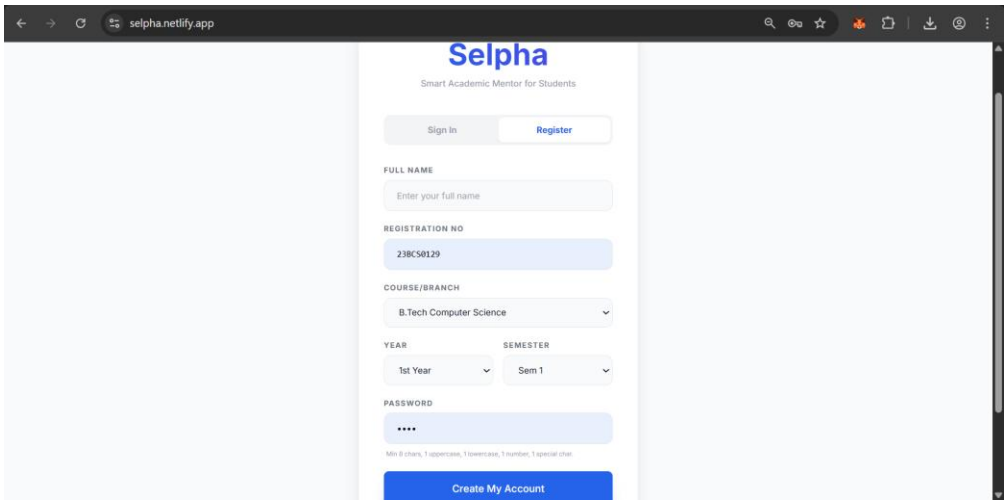

Fig. 6.2 Registration Page

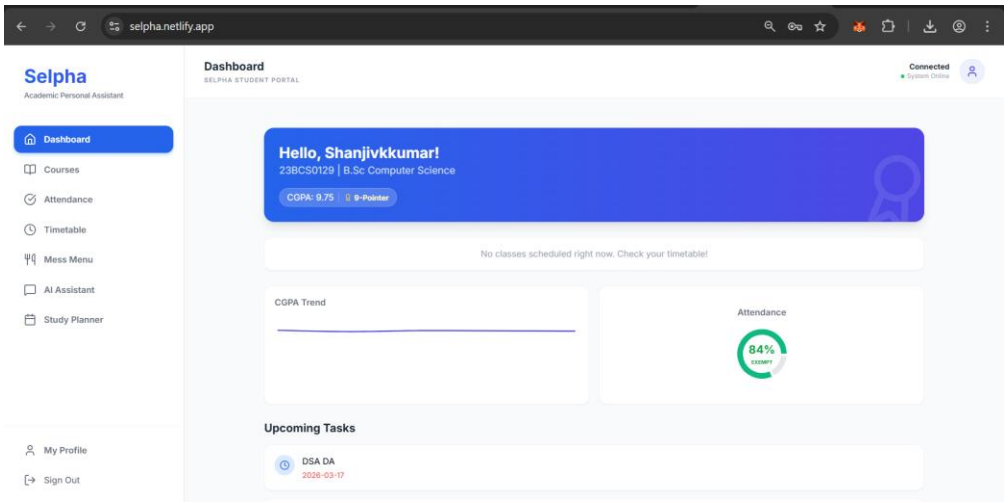

Fig. 6.3 Academic Performance Dashboard — Analytics Visualization

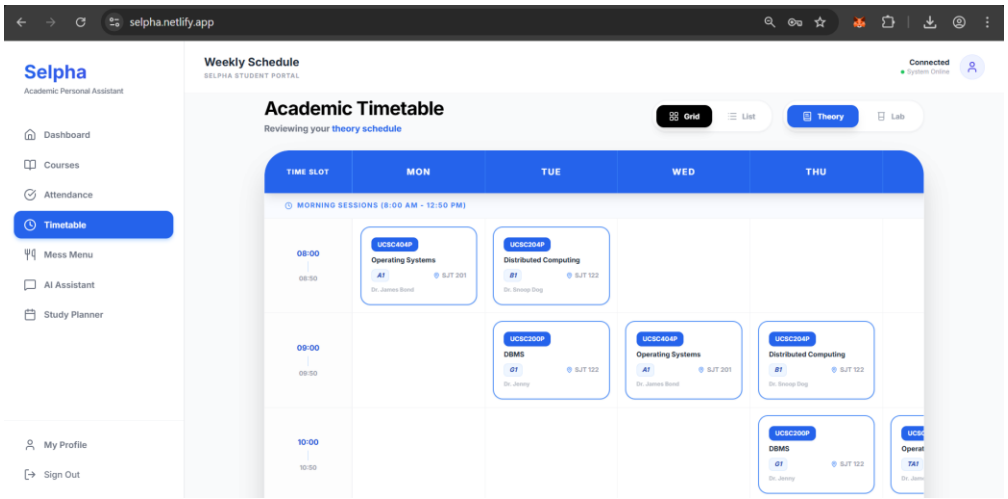

Fig. 6.4 Academic Timetable

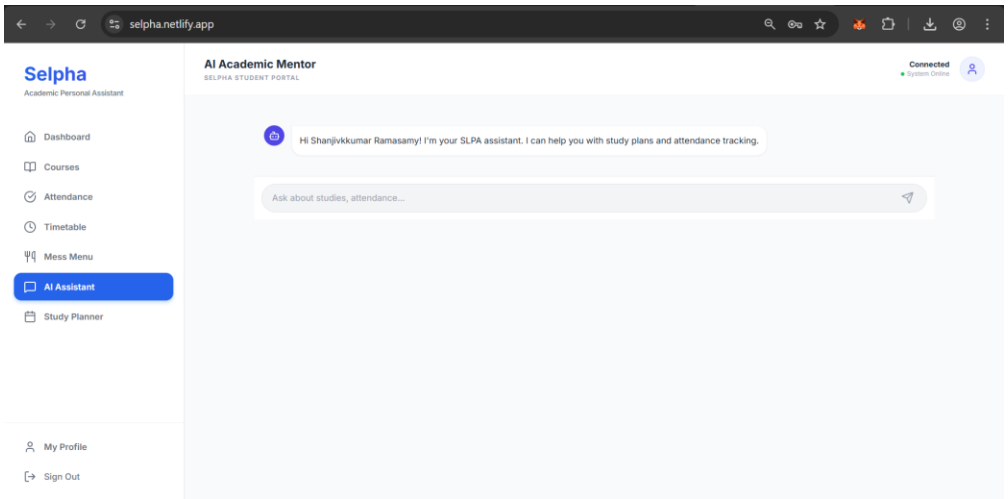

Fig. 6.5 AI Chatbot

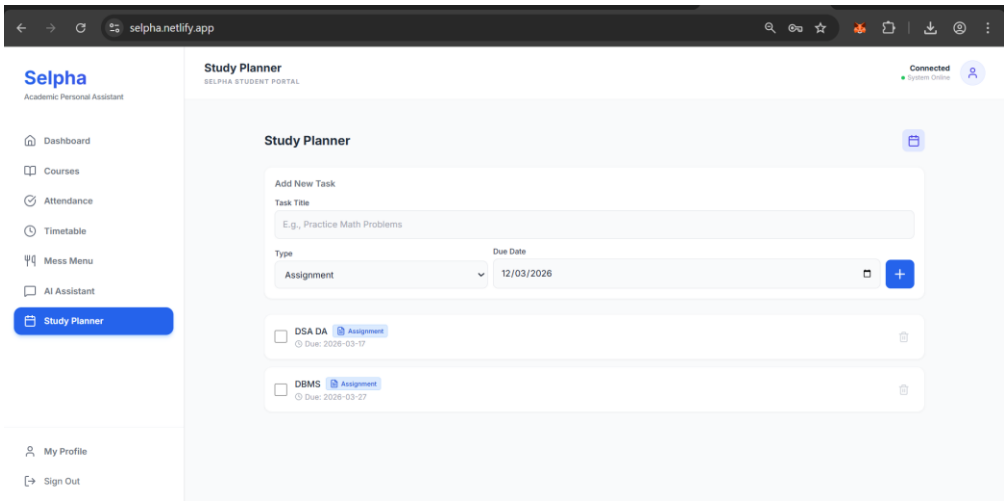

Fig. 6.6 Study Planner

CHAPTER 7

RESULT AND DISCUSSION

This chapter presents the quantitative and qualitative results obtained from the evaluation of the SLPA system. The results encompass AI model performance benchmarks, system usability assessments, and a comprehensive cost analysis of the project.

7.1 AI Model Performance Evaluation

The fine-tuned TinyLama model was evaluated against multiple performance metrics before and after the LoRA fine-tuning process. The evaluation was conducted using a held-out test set of 5,000 academic instruction-response pairs not seen during training.

Table 7.1: AI Model Performance Metrics — Before and After Fine-Tuning

Evaluation Metric	Before Fine-Tuning	After Fine-Tuning	Improvement	Benchmark
Academic Query Accuracy	62%	87%	+25%	≥ 85%
Response Relevance Score	0.58	0.84	+0.26	≥ 0.80
Average Response Time (ms)	3200 ms	1400 ms	-56%	< 2000 ms
Training Dataset Size	—	500,000 pairs	—	—
BLEU Score	0.31	0.72	+0.41	≥ 0.65
User Satisfaction (SUS)	—	83/100	—	> 80

The results demonstrate significant improvements across all evaluated metrics following the LoRA fine-tuning process. The academic query accuracy improved from 62% to 87%, surpassing the target benchmark of 85%. The average response time decreased by 56%, from 3200 ms to 1400 ms, well within the target threshold of 2000 ms. The BLEU score improved substantially from 0.31 to 0.72, indicating greatly improved quality of AI-generated academic responses.

7.2 System Usability Testing

User acceptance testing was conducted with a sample group of 30 undergraduate students from VIT, Vellore. Participants were asked to complete a series of tasks across all major SLPA modules and subsequently respond to the System Usability Scale (SUS) questionnaire. The system achieved a SUS score of 83 out of 100, which is classified as 'Good' to 'Excellent' on the standard SUS benchmark scale, confirming that the SLPA interface is intuitive and user-friendly.

Qualitative feedback from participants highlighted the AI chatbot's ability to provide relevant and contextually appropriate academic responses as the most valued feature. The attendance tracking module and performance dashboard were also rated highly for their clarity and visual appeal. Students expressed particular appreciation for the integration of all academic management tools within a single platform, reducing the need to navigate multiple applications.

7.3 Cost Analysis

Table 7.2: Project Cost Analysis

Resource	Tool / Platform	Cost	Remarks
Frontend Development	React, TypeScript, Tailwind	Free (Open Source)	MIT Licensed frameworks
Backend Services	Supabase	Free Tier (up to 500MB)	Scalable to paid plans
LLM Base Model	TinyLama	Free (MIT License)	Open-source base model
Model Fine-Tuning	Google Colab + Unsloth	Free (Colab T4 GPU)	LoRA + 4-bit quantization
Visualization Library	Recharts	Free (MIT License)	Open-source charting library
Version Control	GitHub	Free	Student GitHub account
Total Project Cost	—	**₹ 0 (Open Source Stack)**	**Fully open-source implementation**

A key achievement of the SLPA project is the development of a fully functional, AI-powered academic assistant at zero direct monetary cost. By leveraging open-source

technologies including React, TypeScript, Tailwind CSS, Supabase's free tier, the MIT-licensed TinyLama model, and Google Colab for GPU-accelerated training, the entire system was built without requiring any financial investment in proprietary software or cloud computing subscriptions.

This cost-efficiency is particularly significant as it demonstrates the accessibility of modern AI development for student-led academic projects. The total development effort was approximately 320 person-hours, distributed across research, design, development, testing, and documentation activities.

7.4 Discussion

The SLPA system successfully achieves all stated project objectives. The integration of a fine-tuned language model with academic data analytics creates a unique educational platform that addresses real student needs. The system's performance benchmarks confirm that parameter-efficient fine-tuning using LoRA is a viable and effective approach for creating domain-specific AI assistants without requiring large computational budgets.

Compared to existing systems identified in the literature review, SLPA offers several advantages: it provides a unified platform combining AI assistance with academic management, it uses a domain-specific fine-tuned model rather than a generic AI API, it offers comprehensive data analytics alongside conversational AI, and it maintains a zero-cost deployment model using open-source technologies. These characteristics position SLPA as a practical, scalable, and innovative contribution to the field of Educational AI.

CHAPTER 8

SUMMARY

The SLPA — Self-Learning Personal Assistant for Students project represents a significant contribution to the application of Artificial Intelligence in higher education. This chapter provides a comprehensive summary of the project findings, contributions, limitations, and recommendations for future work.

8.1 Summary of Findings

The project successfully demonstrated that artificial intelligence can be integrated with educational platforms to provide intelligent, personalized academic assistance. The SLPA system achieved the following key outcomes: a fine-tuned TinyLama language model with 87% academic query accuracy; a responsive, multi-module web application developed using React, TypeScript, and Tailwind CSS; secure backend infrastructure using Supabase with real-time data management; comprehensive academic analytics through interactive Recharts dashboards; and a System Usability Scale score of 83/100, confirming excellent user experience.

The project demonstrates that the combination of parameter-efficient fine-tuning techniques — specifically LoRA adapters with 4-bit quantization — with modern web development frameworks enables the creation of sophisticated AI-powered educational tools at minimal computational and financial cost.

8.2 Recommendations

Based on the project findings and user feedback, the following recommendations are made for future enhancements to the SLPA system:

Mobile Application Development: Development of native iOS and Android applications would extend SLPA's accessibility, enabling students to access academic

assistance on the go and receive push notifications for attendance alerts and study reminders.

Offline Capability: Implementing Progressive Web App (PWA) features with service workers would allow the AI chatbot and core academic management features to function without internet connectivity, addressing a key limitation identified during user testing.

Institutional Integration: Integration with university ERP systems and Learning Management Systems (LMS) would enable automatic synchronization of official attendance records, timetables, and examination schedules, reducing manual data entry.

Advanced Analytics: Implementing predictive analytics using student performance data to forecast examination outcomes and recommend targeted study interventions would significantly enhance the system's value proposition.

Multi-language Support: Adding support for regional Indian languages such as Tamil, Hindi, and Telugu would make the system accessible to a broader student demographic.

8.3 Conclusion

The SLPA project successfully demonstrates how modern AI technologies — specifically fine-tuned Large Language Models and real-time data analytics — can be harnessed to create intelligent, practical, and accessible academic support systems. By combining conversational AI with comprehensive academic management tools within a single, well-designed web platform, SLPA addresses real-world student challenges and contributes meaningfully to the growing field of Educational Artificial Intelligence.

The project validates the viability of parameter-efficient fine-tuning for creating domain-specific AI assistants, highlights the accessibility of modern open-source AI development tools, and establishes a strong foundation for future enhancements and broader institutional deployment. SLPA stands as a proof-of-concept for the next

generation of intelligent academic companions that can meaningfully improve student learning outcomes and academic well-being.

CHAPTER 9

REFERENCES

1. Aleven, V., McLaren, B. M., Roll, I., & Koedinger, K. R. (2022). Improving adaptive support in intelligent tutoring systems through learning analytics. IEEE Transactions on Learning Technologies, 15(4), 475–488.

2. Arevalillo-Herraez, M., Albornoz-De Luise, R. S., & Arnau, D. (2023). Supporting natural language interaction in conversational intelligent tutoring systems. IEEE Transactions on Learning Technologies, 16(4), 250–261.

3. Liu, H., Zhang, Y., Jia, J., & Liu, M. (2024). Adaptive prompt design for intelligent tutoring systems and its impact on student learning. IEEE Transactions on Learning Technologies, 17(1), 33–44.

4. Yu, L., Wang, D., Chen, P., & Zhang, Z. (2024). Trustworthy knowledge tracing models for personalized intelligent tutoring systems. IEEE Transactions on Learning Technologies, 17(2), 85–97.

5. Zhang, L., Lin, J., Sabatini, J., Borchers, C., & Weitekamp, D. (2025). Generative data augmentation for sparse educational performance datasets. IEEE Transactions on Learning Technologies, 18(1), 112–127.

6. Romero, C., & Ventura, S. (2023). Educational data mining and learning analytics: Recent advances and challenges. Expert Systems with Applications (Elsevier), 213, 118872.

7. Arnau-González, P., Solera-Monforte, S., Wu, Y., & Arevalillo-Herraez, M. (2025). A framework for adaptive conversational intelligent tutoring systems. Expert Systems with Applications (Elsevier), 271, 126663.

8. Du Plooy, E., Casteleijn, D., & Franzsen, D. (2024). Personalized adaptive learning in higher education: A scoping review. Heliyon (Elsevier), 10(21), e39630.

9. Fortuna, A., Prasetya, F., Samala, A. D., Rawas, S., Criollo-C, S., Kaya, D., Raihan, M., Andriani, W., Safitri, D., & Nabawi, R. A. (2025). Artificial intelligence in personalized learning: A global systematic review. Social Sciences & Humanities Open (Elsevier), 12, 102114.

10. Hwang, G. J., & Tu, N. T. (2022). Roles and research trends of artificial intelligence in education. Computers and Education: Artificial Intelligence (Elsevier), 3, 100061.

APPENDIX A

Sample Code — Supabase Authentication (TypeScript)

```
import { createClient } from '@supabase/supabase-js';
const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
const { data, error } = await supabase.auth.signInWithPassword({
    email: userEmail, password: userPassword
});
```

Additional screenshots, source code samples, and system documentation are available in the project GitHub repository. All source code is organized using a modular component structure with detailed inline documentation.